

LAB 12

Bypass de la Détection de Root

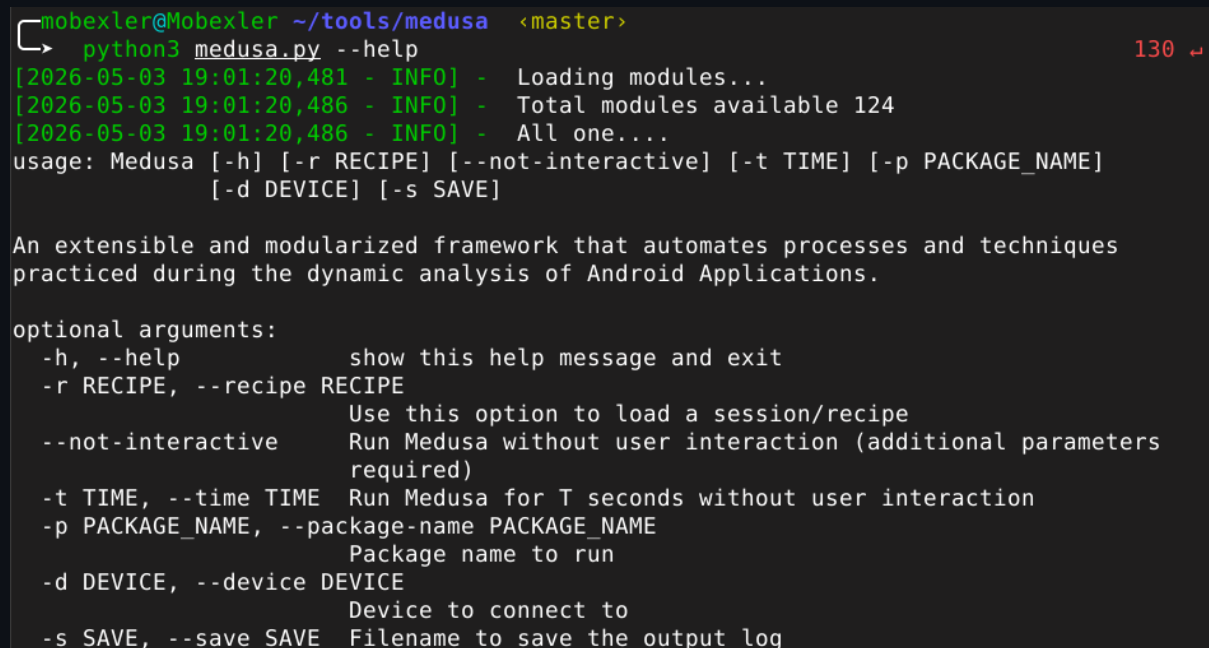
avec Medusa — Android Dynamic Instrumentation

Plateforme	Mobexler (Debian Bullseye)
Émulateur	Genymotion — Android 10 (API 29)
App cible	OWASP UnCrackable Level 1
Outil	Medusa (Ch0pin/medusa) — dev
Frida	16.0.19

Installation de Medusa

Medusa est un framework d'instrumentation dynamique modulaire basé sur Frida, conçu pour automatiser les analyses de sécurité d'applications Android. Il dispose de 124 modules couvrant des scénarios variés allant de la détection de root au bypass SSL en passant par l'interception réseau.

```
git clone https://github.com/Ch0pin/medusa.git ~/tools/medusa
cd ~/tools/medusa
pip3 install -r requirements.txt
python3 medusa.py --help
```



```
mobexler@Mobexler ~/tools/medusa <master>
└─> python3 medusa.py --help 130 ↵
[2026-05-03 19:01:20,481 - INFO] - Loading modules...
[2026-05-03 19:01:20,486 - INFO] - Total modules available 124
[2026-05-03 19:01:20,486 - INFO] - All one...
usage: Medusa [-h] [-r RECIPE] [--not-interactive] [-t TIME] [-p PACKAGE_NAME]
              [-d DEVICE] [-s SAVE]

An extensible and modularized framework that automates processes and techniques
practiced during the dynamic analysis of Android Applications.

optional arguments:
  -h, --help            show this help message and exit
  -r RECIPE, --recipe RECIPE
                        Use this option to load a session/recipe
  --not-interactive      Run Medusa without user interaction (additional parameters
                        required)
  -t TIME, --time TIME  Run Medusa for T seconds without user interaction
  -p PACKAGE_NAME, --package-name PACKAGE_NAME
                        Package name to run
  -d DEVICE, --device DEVICE
                        Device to connect to
  -s SAVE, --save SAVE  Filename to save the output log
```

FIGURE 1 – Medusa installé — 124 modules disponibles, framework opérationnel

Exploration des modules de détection root

Avant de procéder au bypass, la commande `search root` dans la console Medusa révèle quatre modules dédiés à cette problématique. Parmi eux, `root_detection/universal_root_detection_bypass` est le plus complet : il intercepte simultanément la lecture de `Build.TAGS`, les vérifications d'existence des binaires `su` et `busybox`, ainsi que les appels `Runtime.exec()`.

```
(local) medusa> search root
root_detection/jailMonkey_react_native
root_detection/universal_root_detection_bypass
root_detection/rootbeer_detection_bypass_no_obfuscation
root_detection/rootbeer_detection_bypass
```

```

- 3rd party installed applications:

(local) medusa>search root
root_detection/jailMonkey_react_native
root_detection/universal_root_detection_bypass
root_detection/rootbeer_detection_bypass_no_obfuscation
root_detection/rootbeer_detection_bypass
(local) medusa>

```

FIGURE 2 – Résultat de search root — 4 modules de bypass identifiés

Connexion à l'émulateur Genymotion

Medusa énumère automatiquement les appareils disponibles via Frida. L'émulateur Genymotion, connecté via ADB TCP sur l'adresse 169.254.79.101:5555, apparaît sous l'identifiant Phone. Les propriétés système confirment un environnement rooté : `ro.build.tags` retourne `test-keys` au lieu de `release-keys`, ce qui constitue précisément l'un des indicateurs détectés par les applications.

```

MEVUSH (Android) Version: dev

Type help for options

2026-05-03 19:11:55,692 - INFO] - Available devices:
0) Device(id="local", name="Local System", type='local')
1) Device(id="socket", name="Local Socket", type='remote')
2) Device(id="169.254.79.101:5555", name="Phone", type='usb')

Enter the index of the device to use: 2

Device properties:
ro.product.manufacturer]: [Genymobile]
ro.product.name]: [vbox86p]
ro.build.version.all_codenames]: [REL]
ro.build.version.base_os]: []
ro.build.version.codename]: [REL]
ro.build.version.incremental]: [715]
ro.build.version.min_supported_target_sdk]: [23]
ro.build.version.preview_sdk]: [0]
ro.build.version.preview_sdk_fingerprint]: [REL]
ro.build.version.release]: [10]
ro.build.version.sdk]: [29]
ro.build.version.security_patch]: [2020-01-01]
ro.build.id]: [001D.200105.002]
ro.build.tags]: [test-keys]
ro.build.tags.geny-def]: [test-keys]

3rd party installed applications:
0) com.android.insecurebankv2
1) com.withsecure.dz
2) owasp.mstg.uncrackable1
169.254.79.101:5555) medusa>

```

FIGURE 3 – Connexion établie — propriétés système de l'émulateur Genymotion

Chargement du module de bypass

Le module `root_detection/universal_root_detection_bypass` est ajouté à la session active. Medusa maintient une liste de modules courants qui seront compilés et injectés ensemble lors de l'exécution.

```
use root_detection/universal_root_detection_bypass
```

```
(169.254.79.101:5555) medusa>use root_detection/universal_root_detection_bypass

Current Mods:
0) root_detection/universal_root_detection_bypass

(169.254.79.101:5555) medusa>|
```

FIGURE 4 – Module `universal_root_detection_bypass` ajouté à la session

Comportement sans bypass — Root détecté

Sans instrumentation, l'application UnCrackable Level 1 détecte immédiatement l'environnement rooté au démarrage. L'utilisateur ne peut pas accéder aux fonctionnalités de l'application : dès que les vérifications Java identifient des indicateurs de root, l'application force sa fermeture.

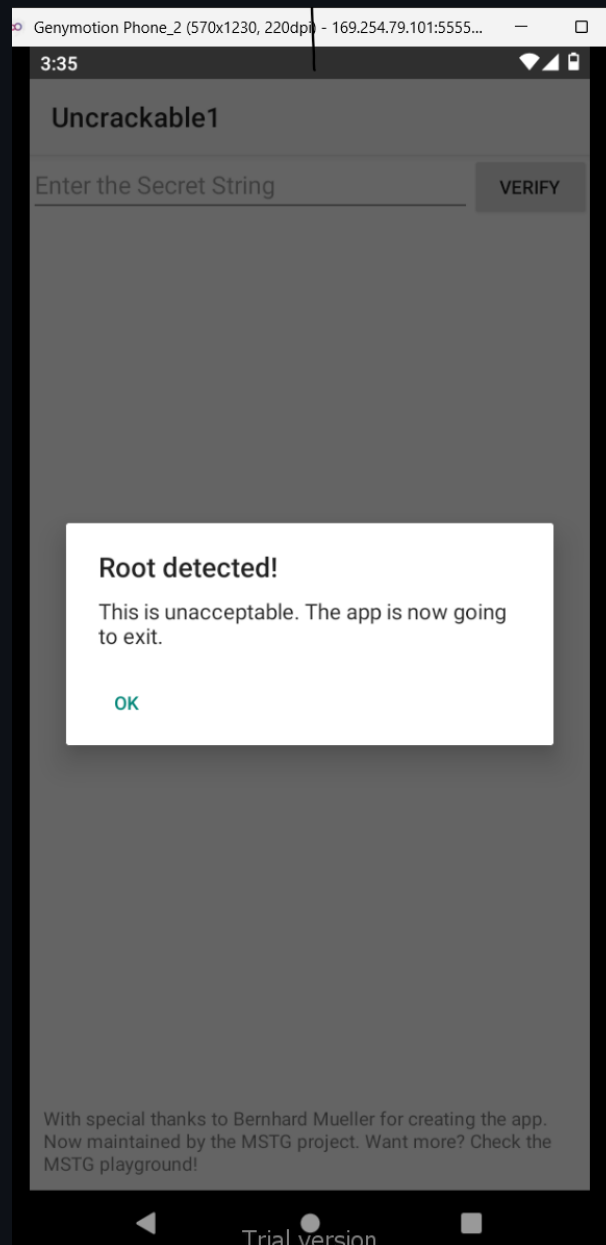


FIGURE 5 – Alerte Root detected! — comportement sans bypass

Injection du module et session active

La commande `run` avec le nom du package cible déclenche l'attachement de la session Frida et l'injection du script compilé. Medusa confirme le chargement du script anti-root avec le message `LOADING ANTI ROOT DETECTION SCRIPT`.

```
run com.android.insecurebankv2
[INFO] Attaching frida session to PID - 7199
-----LOADING ANTI ROOT DETECTION SCRIPT-----
Loaded 9188 classes!
(in-session) type ? for options:>
```

```
(169.254.79.101:5555) medusa>run com.android.insecurebankv2
[2026-05-04 07:32:13,051 - INFO] - Attaching frida session to PID - 7199

-----LOADING ANTI ROOT DETECTION SCRIPT-----
Loaded 9188 classes!
loaded: -1
ProcessManager hook not loaded
KeyInfo hook not loaded
(in-session) type ? for options:>
-----Application Info-----

- Frida version: 16.0.19
- Script runtime: QJS
- Application Name: android.app.Application
- Files Directory: /data/user/0/com.android.insecurebankv2/files
- Cache Directory: /data/user/0/com.android.insecurebankv2/cache
- External Cache Directory: /storage/emulated/0/Android/data/com.android.insecurebankv2/cache
- Code Cache Directory: /data/user/0/com.android.insecurebankv2/code_cache
- Obb Directory: /storage/emulated/0/Android/obb/com.android.insecurebankv2
- Package Code Path: /data/app/com.android.insecurebankv2-Gz1dBcrsgbv2ECRX_wBj9g==/base.apk

-----EOF Application Info-----
```

FIGURE 6 – Session active — script anti-root injecté, 9188 classes chargées

Informations de session

En session active, la commande `i` expose les métadonnées de l'application : répertoires de données, cache, chemins APK.

```
(in-session) type ? for options:>i
Application Name: android.app.Application
Data Directory: /data/user/0/com.android.insecurebankv2/files
Cache Directory: /data/user/0/com.android.insecurebankv2/cache
External Cache Directory: /storage/emulated/0/Android/data/com.android.insecurebankv2/cache
Code Cache Directory: /data/user/0/com.android.insecurebankv2/code_cache
Obb Directory: /storage/emulated/0/Android/obb/com.android.insecurebankv2
Apk Directory: /data/app/com.android.insecurebankv2-Gz1dBcrsgbv2ECRX_wBj9g==/base.apk
```

FIGURE 7 – Informations de session — répertoires et chemins de l'application

Résultat — Application accessible sans alerte

Avec le module de bypass actif, l'application UnCrackable Level 1 s'ouvre sans déclencher l'alerte de détection root. Ce résultat démontre que les vérifications Java de détection root peuvent être neutralisées par instrumentation dynamique sans modifier le code de l'application.

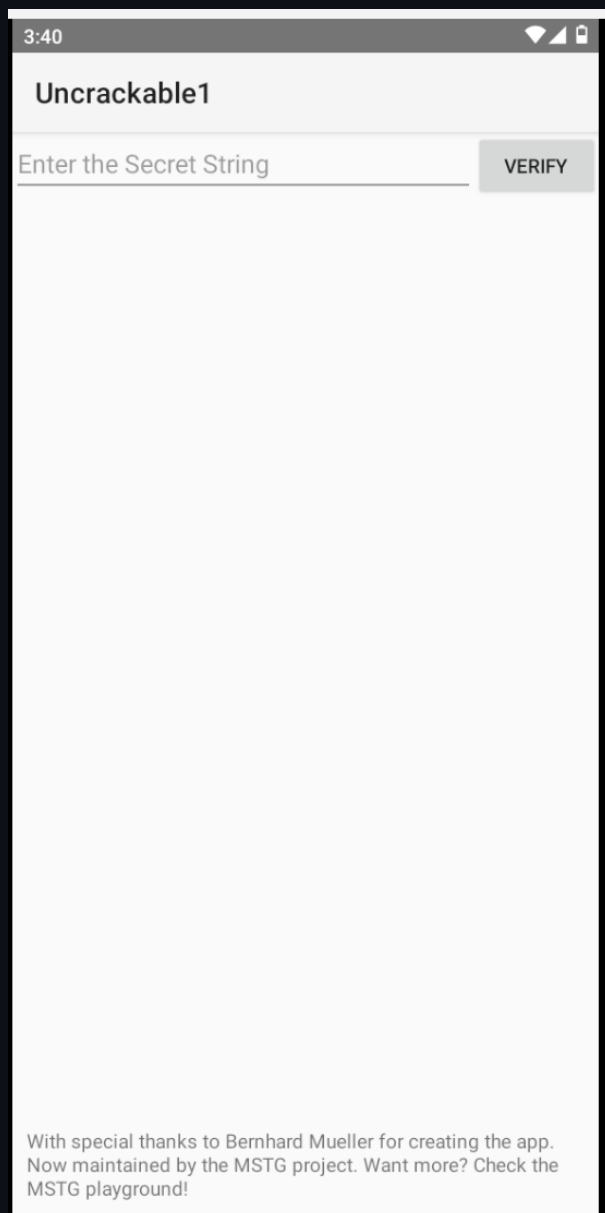


FIGURE 8 – Application ouverte après bypass — aucune alerte root

Bilan des captures

Fichier	Contenu
01-medusa-installed.png	Medusa installé — 124 modules
02-medusa-search-root.png	4 modules root bypass disponibles
03-medusa-connected.png	Connexion à Genymotion établie
04-medusa-module-loaded.png	Module bypass chargé
05-medusa-run.png	Session active — script injecté
06-after-bypass.png	Application ouverte sans alerte
07-root-detected-before.png	Comportement sans bypass
08-medusa-app-info.png	Informations de session